

Introduction to Computer Systems in HPC

Day 1 - The Hidden World Beneath Your Code

Chenxiao Li (@YooLc)

June 29, 2025

Zhejiang University Supercomputing Team

1. Introduction
2. Operating System Basics
3. Process
4. Thread
5. Virtual Memory

Introduction

Introduction

Abstraction

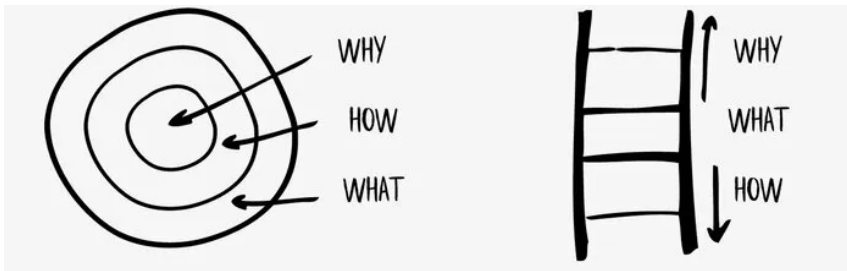
💡 Abstraction: A Great Idea in Computer Architecture

Abstraction is the process of **generalizing concrete details**, away from the study of objects and systems, to **focus attention on details of greater importance**

💡 Abstraction: A Great Idea in Computer Architecture

Abstraction is the process of **generalizing concrete details**, away from the study of objects and systems, to **focus attention on details of greater importance**

Three questions: **Why**, **What** and **How**?



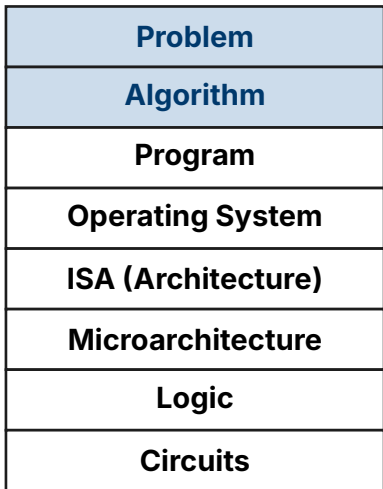
Introduction

Today's Topics

What we will cover today?

Problem
Algorithm
Program
Operating System
ISA (Architecture)
Microarchitecture
Logic
Circuits

What we will cover today?

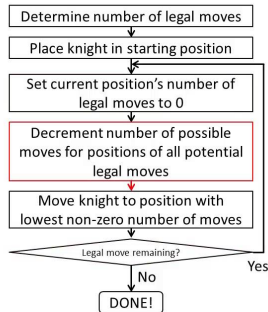


↓ Programming Languages: C, C++ (OOP), Java
Algorithms: FDS, ADS, Operations Research

Task

	a	b	c	d	e	f	g	h
1	2	3	4	4	4	4	3	2
2	3	3	6	6	6	6	4	3
3	4	6	7	8	8	8	6	4
4	4	6	7	8	8	8	6	4
5	4	6	7	7	8	8	6	4
6	4	0	8	8	8	8	6	4
7	3	4	5	5	6	6	4	3
8	0	3	3	4	4	4	3	2

Algorithm



What we will cover today?

Problem
Algorithm (Code)
Program (Executable)
Runtime System
ISA (Architecture)
Microarchitecture
Logic
Circuits

↓ Compiler Principles
↓ GCC, LLVM (clang, Intel OneAPI, Bisheng)



1
oneAPI



What we will cover today?

Problem
Algorithm
Program
Operating System
ISA (Architecture)
Microarchitecture
Logic
Circuits



Operating System / Computer System III



What we will cover today?

Problem

Algorithm

Program

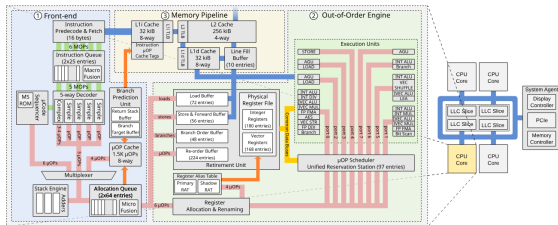
Operating System

ISA (Architecture)

Microarchitecture

Logic

Circuits



Computer Architecture & Computer Organization
Computer System II

What we will cover today?

Problem
Algorithm
Program
Operating System
ISA (Architecture)
Microarchitecture
Logic
Circuits



Digital Logic Design / Computer System I

What we will cover today?

Problem
Algorithm
Program
Operating System
ISA (Architecture)
Microarchitecture
Logic
Circuits



Our Focus in

Introduction to Computer Systems in HPC

Day 1: **Operating System (TLP)**

Day 2: **Computer Architecture (DLP)**

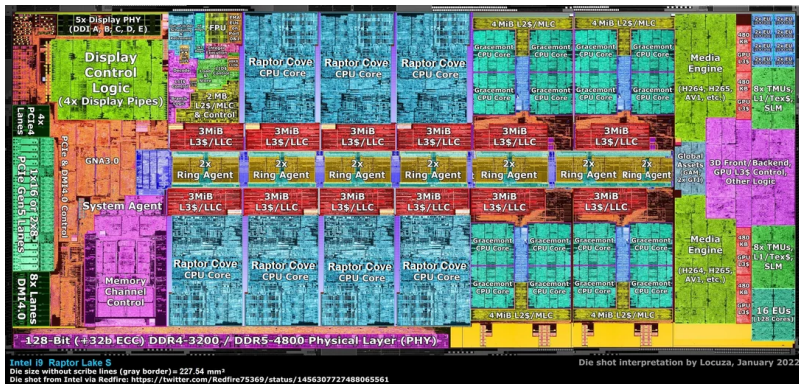
Operating System Basics

Operating System Basics

Introduction

Abstraction of CPU

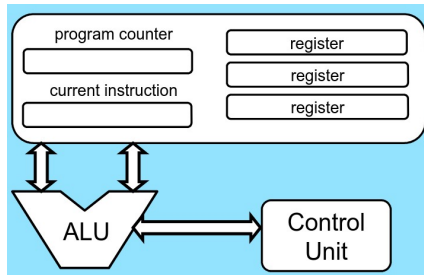
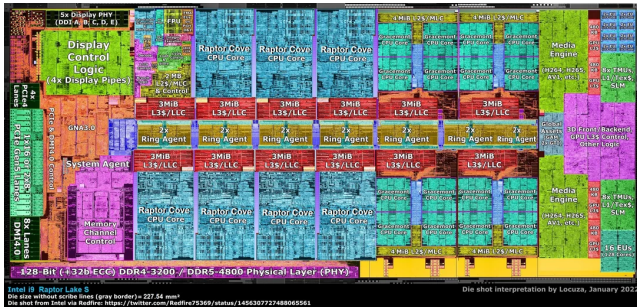
Take a closer look at the CPU die:



Intel Core i9 13900K Die Shot

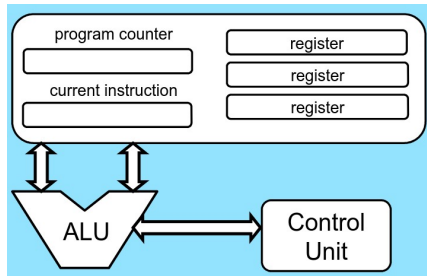
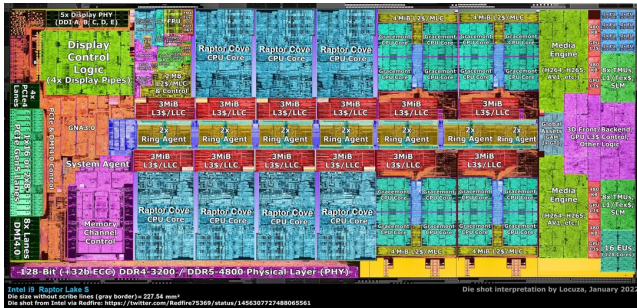
Abstraction of CPU

Take a closer look at the CPU die:



Abstraction of CPU

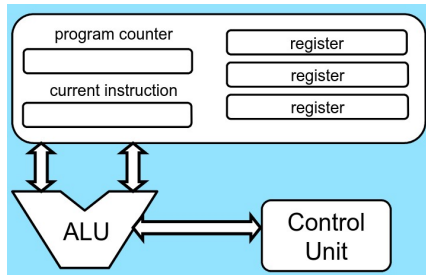
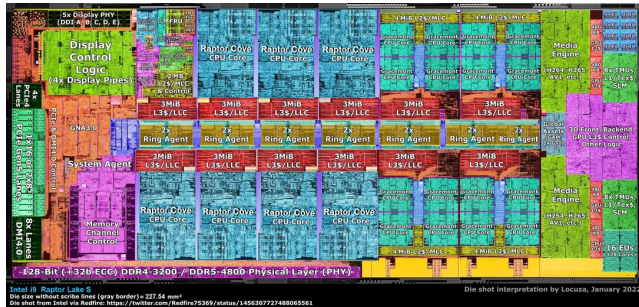
Take a closer look at the CPU die:



- **Program Counter**

Abstraction of CPU

Take a closer look at the CPU die:

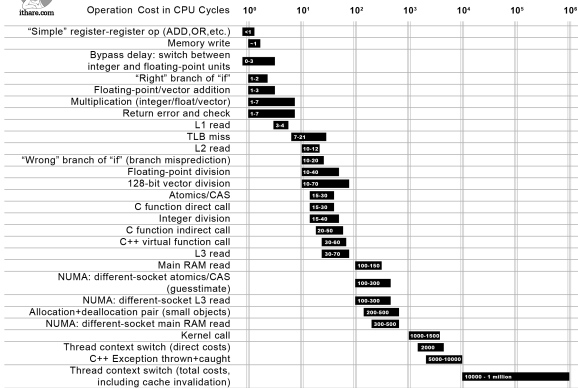


- Program Counter
- Register: stores data processed by the CPU (**a.k.a. context**)

Memory Hierarchy



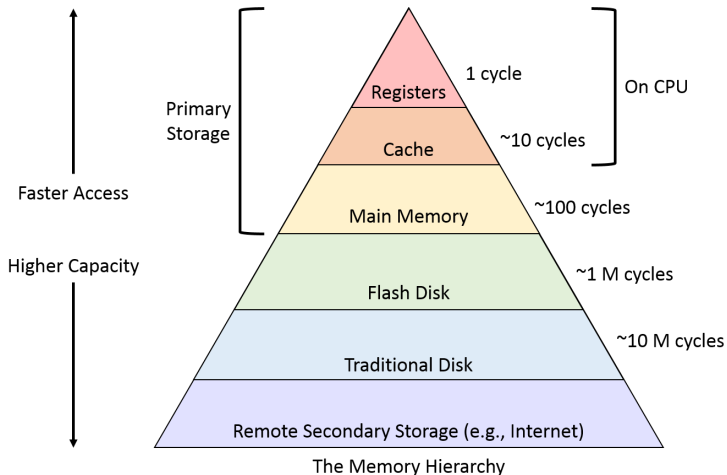
Not all CPU operations are created equal



Distance which light travels while the operation is performed



Memory Hierarchy



🤔 What is an Operating System?

Common operating systems we see in 2025:



Linux



HarmonyOS



- Windows
- macOS, iOS
- Linux, Android

🤔 What is an Operating System?

Common operating systems we see in 2025:



Linux



HarmonyOS



- Windows: **NT**
- macOS, iOS: **Darwin**
- Linux, Android: **Linux**

🤔 What is an Operating System?

Common operating systems we see in 2025:



Linux



HarmonyOS



- Windows
- macOS, iOS
- Linux, Android

★ Operating System is **resource abstractor and resource allocator**.

10k meter view of kernel code

```
void processEvent(event) {  
    switch (event.type) {  
        case NETWORK_COMMUNICATION:  
            NetworkManager.handleEvent(event);  
            break;  
        case SEGMENTATION_FAULT:  
        case INVALID_MODE:  
            ProcessManager.handleEvent(event);  
            break;  
        // ...  
    }  
}
```

- Event
- Handler

Operating System Basics

OS Events

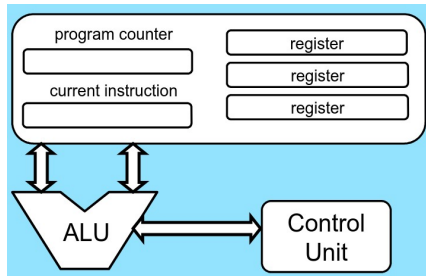
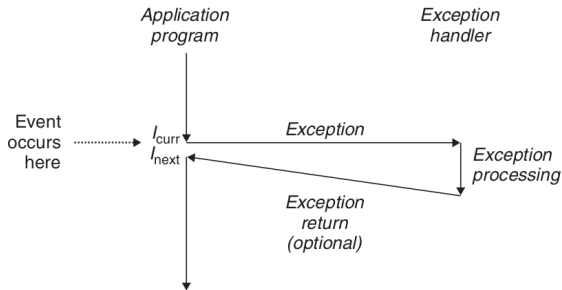
There are two kinds of events, we call them **Traps** in RISC-V¹:

- **Interrupts:** external asynchronous events
 - Hardware-generated
 - e.g., some device controller says “something happened”
- **Exceptions:** unusual condition occurring at instruction run time
 - Software-generated
 - e.g. a division by zero

¹1.6. Exceptions, Traps, and Interrupts, RISC-V Unprivileged ISA Specification

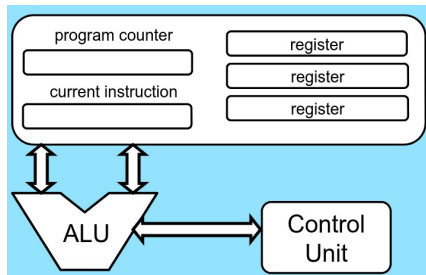
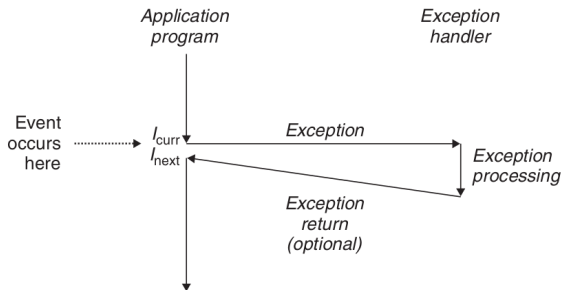
Trapped Control Flow

When a trap occurs, the control flow is switched to its handler:



Trapped Control Flow

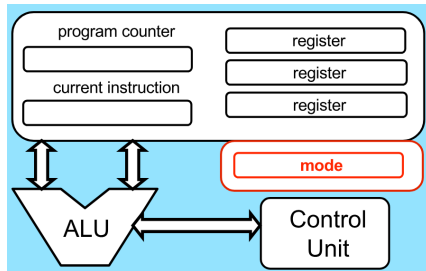
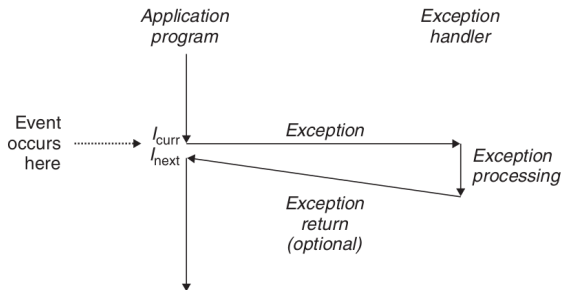
When a trap occurs, the control flow is switched to its handler:



🤔 Q: What do we need to separate the user and kernel space?

Trapped Control Flow

When a trap occurs, the control flow is switched to its handler:

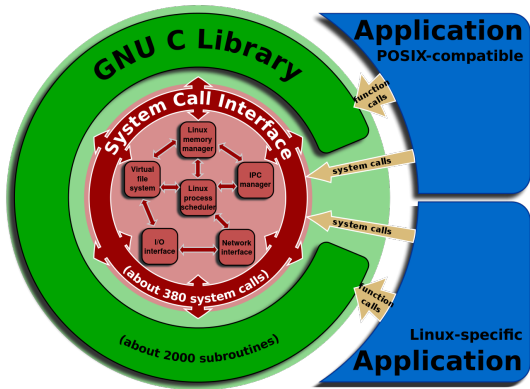


🤔 Q: What do we need to separate the user and kernel space?

😊 A: Add a **Mode Bit** to keep track of the current **privilege level**.

System Calls

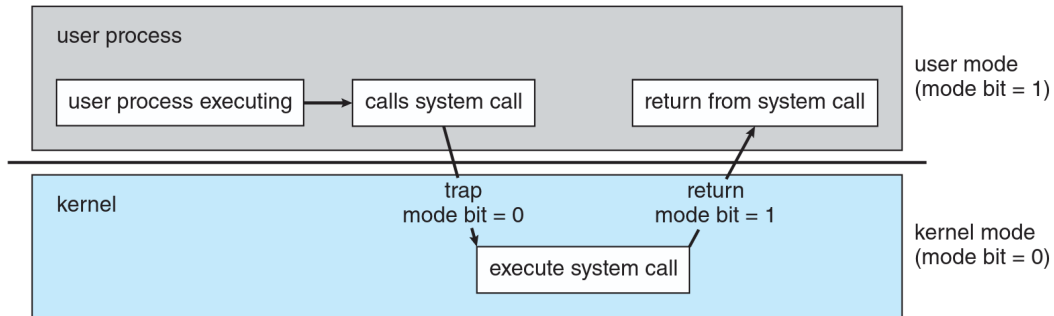
When a user program needs to do something privileged, it **calls a system call**. A system call is a special kind of trap.



10k meter view of kernel code (revised)

```
void processEvent(event) {  
    switch (event.type) {  
        case NETWORK_COMMUNICATION:  
            NetworkManager.handleEvent(event);  
            break;  
        case SEGMENTATION_FAULT:  
        case INVALID_MODE:  
            ProcessManager.handleEvent(event);  
            break;  
        case SYSTEM_CALL:  
            SystemCallManager.handleEvent(event);  
            break;  
        // ...  
    }  
}
```

System Call Handler



- The OS must keep control of the CPU
 - Programs cannot get stuck in infinite loop and lock up the computer
 - Programs cannot gain an unfair share of the computer
- One way in which the kernel retrieves control is when an interrupt occurs
- To make sure that an interrupt will occur reasonably soon, we can use a timer

10k meter view of kernel code (revised)

```
void processEvent(event) {  
    switch (event.type) {  
        case TIMER_INTERRUPT:  
            TimerManager.handleEvent(event);  
            break;  
        // ...  
        case SYSTEM_CALL:  
            SystemCallManager.handleEvent(event);  
            break;  
        // ...  
    }  
}
```

Process

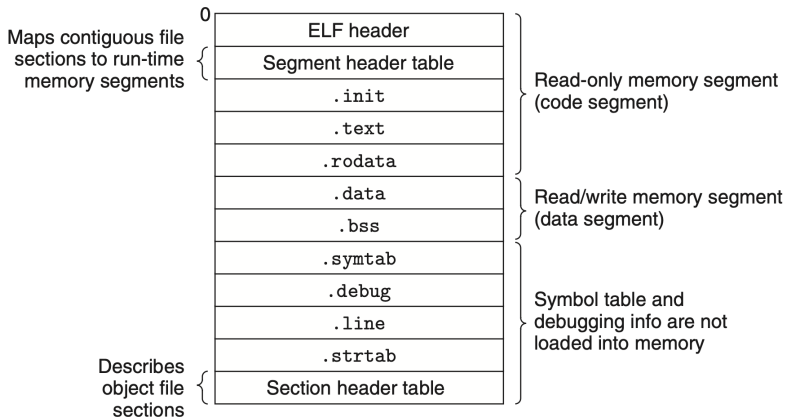
🤔 What's inside a program?

★ A process is **a program in execution, a unit of resource allocation and protection.**

Let's see what makes up a program:

```
$ gcc hello.c -o hello
$ ./hello
Hello World!
$ file hello
hello: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=c6fe8ac2e5be3c78cf31179a5cb31df3978ca160,
for GNU/Linux 3.2.0, not stripped
```

ELF: Executable and Linkable Format

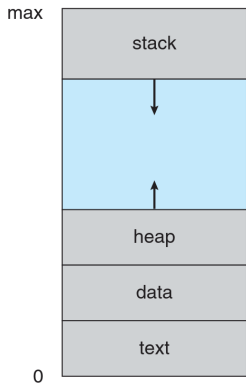


ELFs are organized into sections

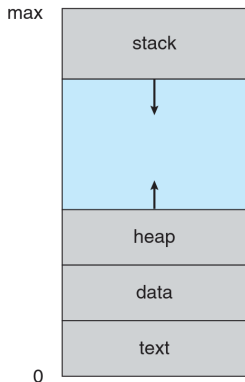
Process Address Space

A Process = A Program in Execution =

- **In Memory (Unique Address Space):**
 - **Code:** Text section, initially stored on disk



Process Address Space

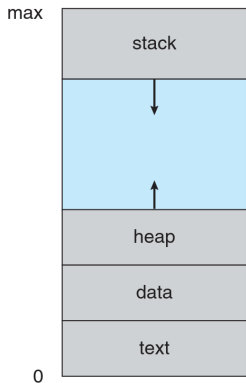


A Process = A Program in Execution =

- **In Memory (Unique Address Space):**

- **Code:** Text section, initially stored on disk
- **Data Section:**
 - **BSS:** Uninitialized data
 - **Data:** Initialized data

Process Address Space

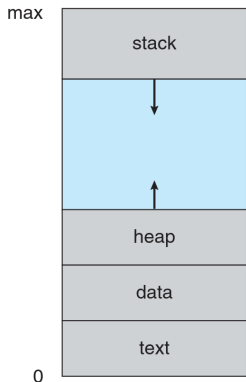


A Process = A Program in Execution =

- **In Memory (Unique Address Space):**

- **Code:** Text section, initially stored on disk
- **Data Section:**
 - **BSS:** Uninitialized data
 - **Data:** Initialized data
- **Stack:** Function frames, local variables, etc.
- **Heap:** Dynamic memory allocation

Process Address Space



A Process = A Program in Execution =

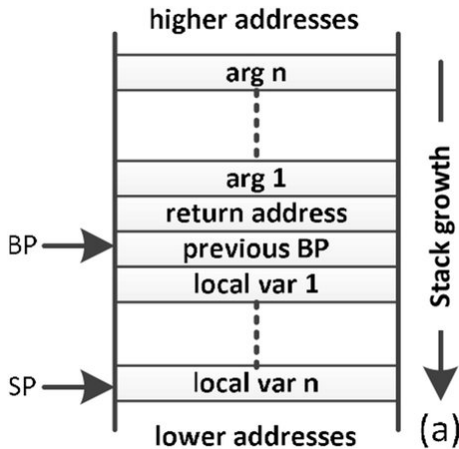
- **In Memory (Unique Address Space):**

- **Code:** Text section, initially stored on disk
- **Data Section:**
 - **BSS:** Uninitialized data
 - **Data:** Initialized data
- **Stack:** Function frames, local variables, etc.
- **Heap:** Dynamic memory allocation

- **Context:**

- **Program Counter:** Points to the next instruction to execute (i.e., an address in the code)
- Content of the processor's **registers**

Stack Frame



Function prologue code:

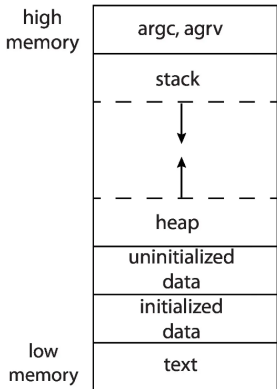
```
1: push  rbp
2: mov   rsp, rbp
3: sub   rsp, N      (b)
```

Function epilogue code:

```
1: mov   rbp, rsp
2: pop   rbp
3: ret      (c)
```

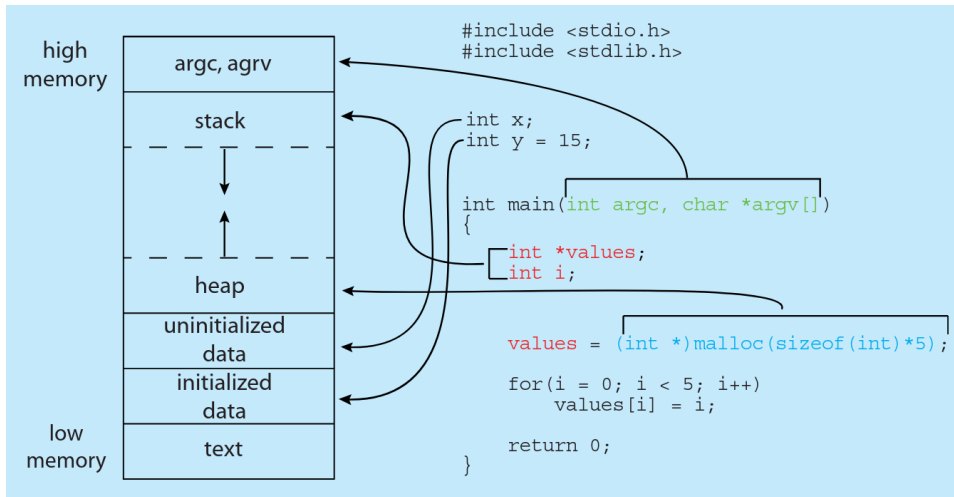
Quick Check: Memory Layout of C Program

Match the variables to the correct memory location:



```
#include <stdio.h>
#include <stdlib.h>
int x;
int y = 15;
int main(int argc, char *argv[]) {
    int *values;
    int i;
    values = (int *)malloc(sizeof(int) * 5);
    for (i = 0; i < 5; i++)
        values[i] = i;
    return 0;
}
```

Quick Check: Memory Layout of C Program (Answer)



Process

Process Schedule

- **Non-preemptive:** The process must finish its execution before the OS can schedule another process.

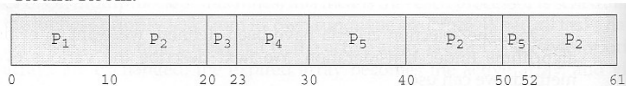
Process Schedule

- **Non-preemptive:** The process must finish its execution before the OS can schedule another process.
- **Preemptive:** The OS can interrupt a process to schedule another one.

Process Schedule

- **Non-preemptive:** The process must finish its execution before the OS can schedule another process.
- **Preemptive:** The OS can interrupt a process to schedule another one.
 - Round Robin
 - Priority-based

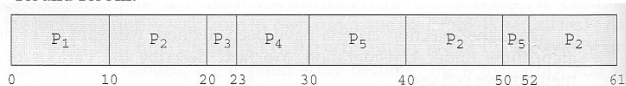
Round Robin:



Process Schedule

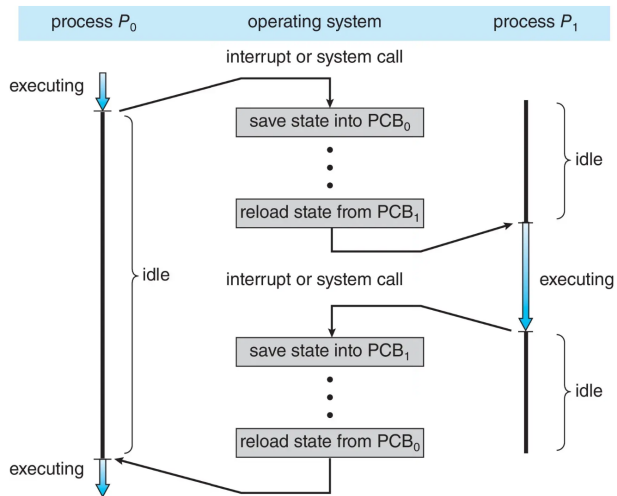
- **Non-preemptive:** The process must finish its execution before the OS can schedule another process.
- **Preemptive:** The OS can interrupt a process to schedule another one.
 - Round Robin
 - Priority-based

Round Robin:



- ★ **Also decide which process to run on which core.** (Control by **Core Affinity**)

Process Switch



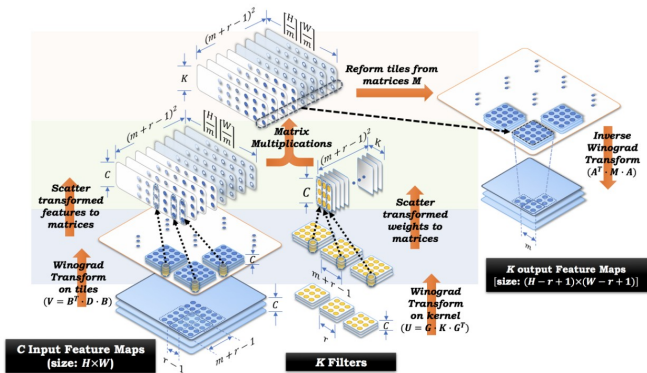
Process

Inter-Process Communication (IPC)

Inter - Process Communication

Why?

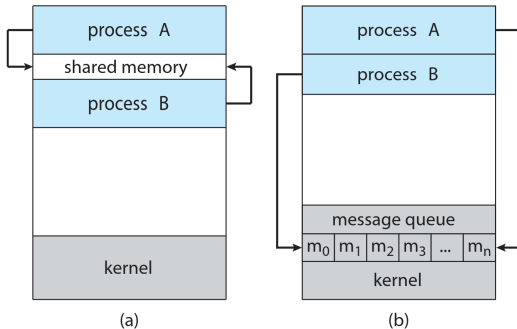
- Processes within a host may be **cooperating**



Inter - Process Communication

How?

- **Message Passing:** We use **MPI** to send / receive messages.
- Shared Memory, Signal, Pipe, Socket...



Disadvantages of IPC: **It's too heavy!** 🙄

- Due to **isolation**, processes do not share address space.

Disadvantages of IPC: **It's too heavy!** 🙄

- Due to **isolation**, processes do not share address space.
- For message passing:
 - Need to maintain a **message queue** for each process.
 - Data need to be **copied between address spaces**.

Disadvantages of IPC: **It's too heavy!** 🤔

- Due to **isolation**, processes do not share address space.
- For message passing:
 - Need to maintain a **message queue** for each process.
 - Data need to be **copied between address spaces**.
- For shared memory:
 - Synchronization is needed
 - It's **complex for OS to manage**.

Disadvantages of IPC: **It's too heavy!** 🤔

- Due to **isolation**, processes do not share address space.
- For message passing:
 - Need to maintain a **message queue** for each process.
 - Data need to be **copied between address spaces**.
- For shared memory:
 - Synchronization is needed
 - It's **complex for OS to manage**.
- Must flush cache when switching between processes. (Why?)

Disadvantages of IPC: **It's too heavy!** 🙄

- Due to **isolation**, processes do not share address space.
- For message passing:
 - Need to maintain a **message queue** for each process.
 - Data need to be **copied between address spaces**.
- For shared memory:
 - Synchronization is needed
 - It's **complex for OS to manage**.
- Must flush cache when switching between processes. (Why?)

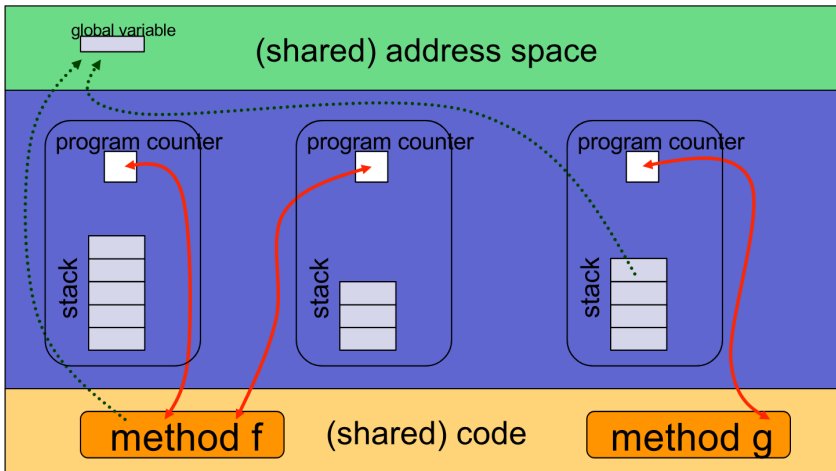
🤔 Is there a better way?

Thread

Thread: Light - Weight Process

- ★ A thread is a **basic unit of execution** within a process. Each thread has its own:
 - Thread ID
 - Program counter
 - Register set
 - Stack
- Shares the following with other threads within the same process:
 - Code section
 - Data section
 - The heap (dynamically allocated memory)
 - Open files and signals
- **Concurrency**: A multi-threaded process **can do multiple things at once**

Thread: Light - Weight Process



Virtual Memory

🎯 Why do we need virtual memory?

- Virtual memory is another abstraction layer between process and physical memory.
 - **Isolation:** Each process has its own address space.

🎯 Why do we need virtual memory?

- Virtual memory is another abstraction layer between process and physical memory.
 - **Isolation:** Each process has its own address space.
 - **Protection:** Deny the process from accessing other processes' or privileged memory.

🎯 Why do we need virtual memory?

- Virtual memory is another abstraction layer between process and physical memory.
 - **Isolation:** Each process has its own address space.
 - **Protection:** Deny the process from accessing other processes' or privileged memory.
 - **Efficiency:** Support Copy-on-Write, can run a partially loaded program.

🎯 Why do we need virtual memory?

- Virtual memory is another abstraction layer between process and physical memory.
 - **Isolation**: Each process has its own address space.
 - **Protection**: Deny the process from accessing other processes' or privileged memory.
 - **Efficiency**: Support Copy-on-Write, can run a partially loaded program.

🤔 How to implement virtual memory?

- We'll only talk about **Paging** today.

- **Page and Frame**
 - Physical memory is split into **fixed-size frames**.
 - Virtual memory is split into **fixed-size pages**.

- **Page and Frame**
 - Physical memory is split into **fixed-size frames**.
 - Virtual memory is split into **fixed-size pages**.
- **Memory Mapping**
 - Hardware support: **MMU** (Memory Management Unit)
 - Function: Translate **virtual page** to **physical frame**
 - Data structure: **Page Table**

- **Page and Frame**
 - Physical memory is split into **fixed-size frames**.
 - Virtual memory is split into **fixed-size pages**.
- **Memory Mapping**
 - Hardware support: **MMU** (Memory Management Unit)
 - Function: Translate **virtual page** to **physical frame**
 - Data structure: **Page Table**
- **Page Fault**
 - If a page is not in physical memory, a page fault occurs.
 - The OS will load the page from disk to physical memory, and update the page table.

Paging in practice

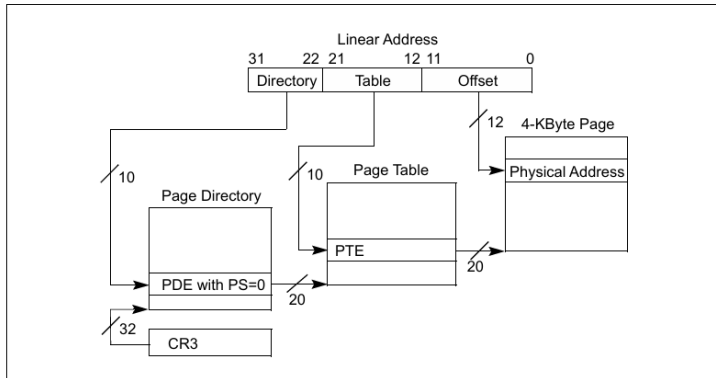
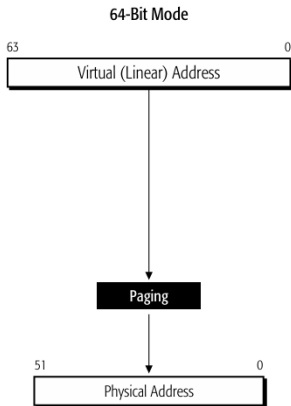


Figure 5-2. Linear-Address Translation to a 4-KByte Page using 32-Bit Paging

- ★ Important concepts that you should remember today:
- Operating System: **a resource abstractor and resource allocator.**
 - Process: **a unit of resource allocation and protection.**
 - Thread: **basic unit of execution** within a process.
 - Others: Memory Hierarchy, Stack Frame, etc.

THANK You

ANY QUESTIONS?